CH09

- demend paging

필요할때 page를 메모리로 가져옴.

lazy swapper (pager) 페이지가 필요없으면 절대 swap X

valid bit

v : in memory

i : not in memory → page fault

- page fault

빈 프레임 찾기, page swap, page table 재설정, valid bit v, 재개

(locality of reference)

- 명령어 재시작 문제

블록 이동시 문제 발생 ← 사전검사 (microcode)

백업 (임시 레지스터)

- 성능

page fault rate p

$(1-p)$(메모리 엑세스) $+ p$(page fault + swap in + swap out)

- COW (copy on write)

부모에서 자식 프로세스 생성시 일단 같은 메모리 공유. ⇒ fork()

이후 수정한 페이지만 복사

(부모중단 ← vfork()
exec() 실행시 유용
vfork는 COW X)

- Page replacement (빈 페이지 부족시)

빈 프레임 없으면, victim frame 선택
dirty 이면 disk에 기록

① FIFO

frame 증가가 page fault 증가시킬수 있음
(belady's anomaly)

② optimal

미래를 보고 가장 오랫동안 사용되지 않을 페이지 교체

③ LRU (least recently used)

과거를 보고 가장 오랫동안 사용되지 않은 페이지 교체

구현

(카운터 , clock 활용
이중 링크 스택 , 참조된것을 맨 위로 업데이트 , 교체시엔 검색 X

LRU와 optimal은 belady anomaly X = frame 증가하면 무조건 page fault↓

- LRU 근사

reference bit 참조하면 1로 표시

second chance 참조안하는애들을 reference bit 0으로 교체
(reference bit 0을 만날때까지)

enhanced
(reference, modify)
(0,0) 교체우선순위
(0,1)
(1,0)
(1,1) ↓

- counting algorithm
참조 횟수를 카운트
  LFU
  MFU

- page buffering
  - 고정할당 fixed
    - equal
    - proportional : 프로세스 크기 비율대로

  - 우선순위할당 priority
    - 자신 프레임 안에서 선택 (local)
    - 낮은 우선순위 중에서 선택 (global)

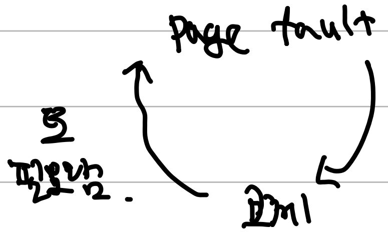
- Non Uniform memory access NUMA (중요 없어짐)
물리적으로 가까운 메모리 선택

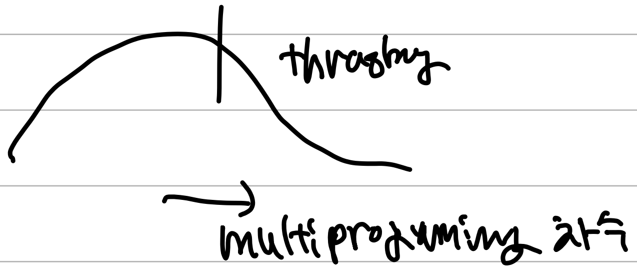
— Thrashing

: 메모리 부족으로 인한 성능저하

↓ CPU utilization

Swapping 하느라 바쁨

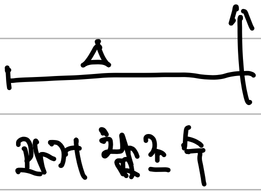

page fault
교체
또 필요함.



Demanding paging : locality 이용

Σ size of locality > 전체 메모리 크기 = thrashing

— Working set model

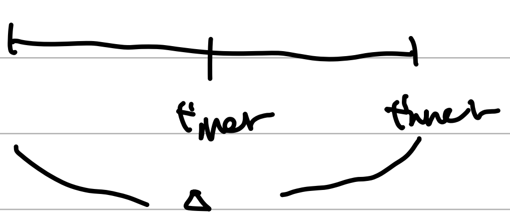


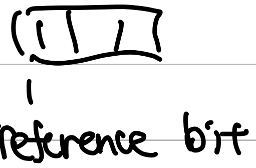
WSS_i (프로세스 i의 working set size)

$D = \Sigma WSS_i$ 전체 필요한 frame 수

$D >$ 총 프레임 수 ⇒ thrashing

근사 : 타이머 인터럽트 사용

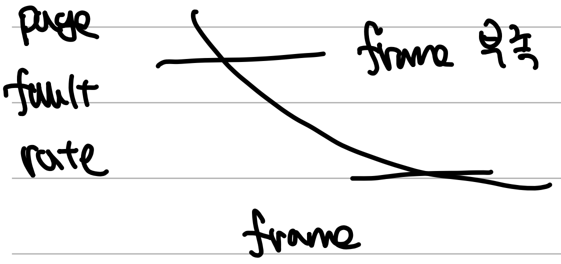

timer
timer
Δ


reference bit

하나만 1이면

working set

- Page fault frequency PFF



frame 부족 (upper bound : 프로세스에 frame 추가)

너무 여유로움 (lower bound : 프로세스 frame 감소)